Coin Counting and Classification System

Computer Vision Course - Final Project Report

Team Member	Role
Mariem Helmy Mostafa (932230189)	Preprocessing: grayscale conversion, contrast enhancement, smoothing
Alaa Ibrahim Abdo (932230031)	Thresholding, morphology, and connected components
Khadiga Mahmoud Ahmed (932230060)	Coin detection, contour filtering, and final visualization
Marise Joseph Melad (9322300141)	Classification, evaluation table, analysis, and report

1. Introduction

Coin counting is an important computer vision application used in automatic cash handling systems, vending machines, banking systems, and coin sorting machines. Counting coins manually can be time-consuming and may lead to errors, especially when many coins are placed close to each other.

The goal of this project is to build an image processing system that can automatically detect coins from an input image, count the total number of detected coins, classify them based on their size, and calculate an estimated total value. The project applies classical image processing techniques such as grayscale conversion, contrast enhancement, smoothing, thresholding, morphological operations, connected components, and contour detection.

2. Methodology

The proposed system follows a step-by-step image processing pipeline. Each stage prepares the image for the next stage and improves the ability of the system to detect coin-like regions.

1. Load the dataset and CSV file.
2. Select 15 evaluation images covering easy, medium, and challenging cases.
3. Convert images to grayscale.
4. Improve contrast using histogram equalization and CLAHE.
5. Apply smoothing filters to reduce noise.
6. Apply thresholding to separate coins from the background.
7. Apply morphological operations to remove noise and fill gaps.
8. Visualize connected components.
9. Detect coin regions using contours.
10. Filter detected regions based on area, circularity, radius, and aspect ratio.
11. Classify coins based on contour area and calculate the total value.
12. Evaluate results using actual counts from the CSV file.
13. Display failure cases and discuss limitations.
14. Test the system using a simple GUI.

3. Implementation Details

Library	Purpose
OpenCV	Image processing, thresholding, morphology, contour detection, drawing output
NumPy	Numerical calculations and array operations
Pandas	Reading and handling the CSV file
Matplotlib	Displaying images, charts, and visualizations
KaggleHub	Downloading the dataset
ipywidgets	Creating the GUI in Google Colab

4. Code Implementation

4.1 Import Libraries

```
!pip install kagglehub -q

import os
import cv2
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import kagglehub

plt.rcParams["figure.figsize"] = (12, 8)
print("Libraries imported successfully.")
```

This code imports the required libraries. OpenCV is used for computer vision operations, NumPy for numerical processing, Pandas for reading the CSV file, and Matplotlib for visualization.

4.2 Download Dataset and Load CSV

```
path = kagglehub.dataset_download("balabaskar/count-coins-image-dataset")

base_path = "/kaggle/input/count-coins-image-dataset"
csv_path = os.path.join(base_path, "coins_count_values.csv")
image_path = os.path.join(base_path, "coins_images", "coins_images", "all_coins")

coins_df = pd.read_csv(csv_path)
images = [img for img in os.listdir(image_path) if img.lower().endswith((".jpg", ".jpeg", ".png"))]

print("CSV Columns:")
print(coins_df.columns)
display(coins_df.head())
print("Total images in all_coins folder:", len(images))
```

This code downloads the dataset, defines the CSV path and image folder path, loads the CSV file, and reads all valid image names from the all_coins folder.

4.3 Select Evaluation Images

```
coins_df = coins_df[coins_df["image_name"].isin(images)].copy()

selected_images = [
    "f14da4ee62.jpg", "01207e3d7e.jpg", "767455e3f9.jpg",
    "2c8f0e58fb.jpg", "3a31d6cc15.jpg",
    "962d332c3c.jpg", "8e89dfa19b.jpg", "207092513f.jpg",
    "44e195e1ef.jpg", "85e4b42fb6.jpg", "b65bf338f7.jpg",
    "f5fd0e7e9b.jpg",
    "0bcb8457d5.jpg", "f35a790bd0.jpg", "c99b8f0e24.jpg"
]

selected_df = coins_df[coins_df["image_name"].isin(selected_images)].copy()
selected_df["order"] = selected_df["image_name"].apply(lambda x: selected_images.index(x))
selected_df = selected_df.sort_values("order").drop(columns=["order"]).reset_index(drop=True)

display(selected_df[["image_name", "coins_count"]])
```

This code selects 15 images for evaluation. The selected set includes easy, medium, and challenging cases. Very difficult images are discussed separately as failure cases.

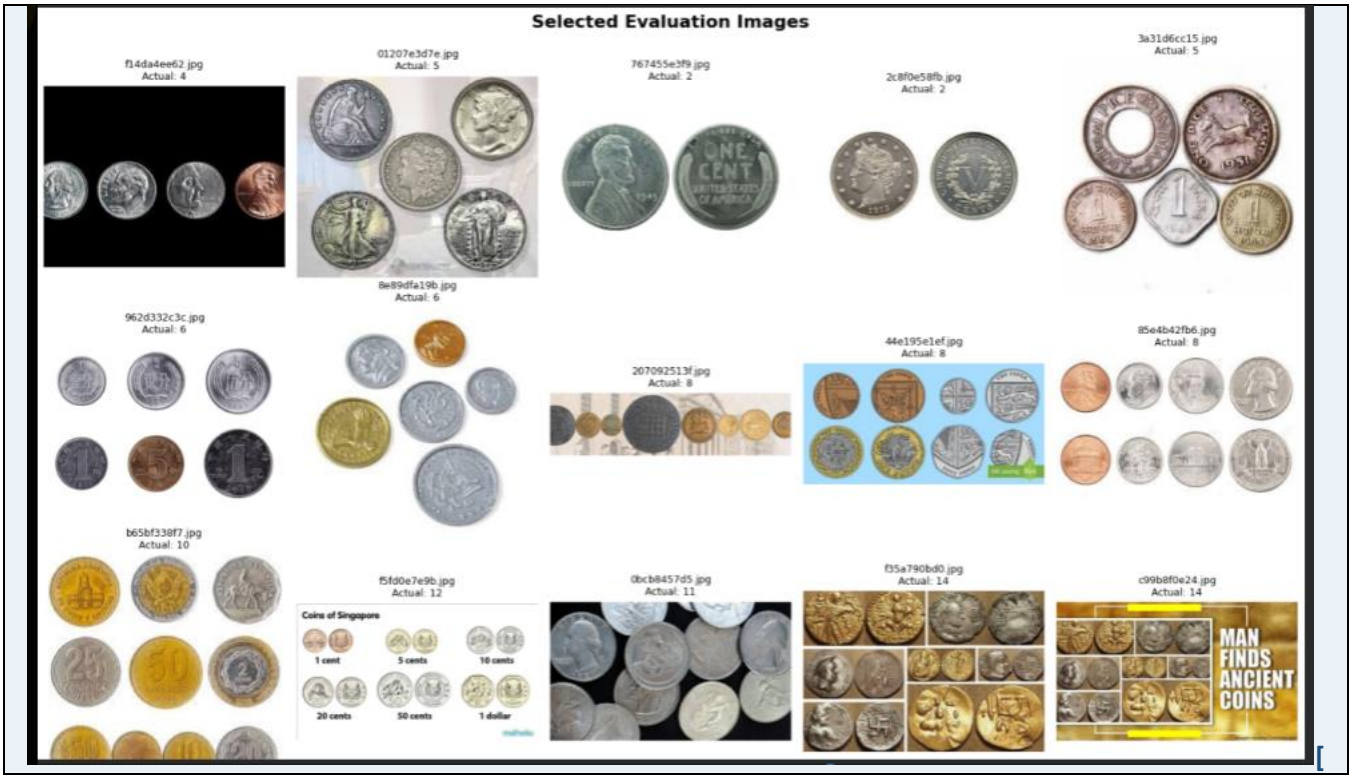


Figure: Selected Evaluation Images

4.4 Grayscale Conversion

```
plt.figure(figsize=(16, 10))

for i, img_name in enumerate(selected_images[:10]):
    img_path = os.path.join(image_path, img_name)
    img = cv2.imread(img_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    plt.subplot(2, 5, i + 1)
    plt.imshow(gray, cmap="gray")
    plt.title(f"Gray {i+1}", fontsize=10)
    plt.axis("off")

plt.suptitle("Grayscale Images", fontsize=16, fontweight="bold")
plt.tight_layout()
plt.show()
```

Grayscale conversion reduces the image from three color channels to one intensity channel, making later steps such as thresholding and contour detection easier.

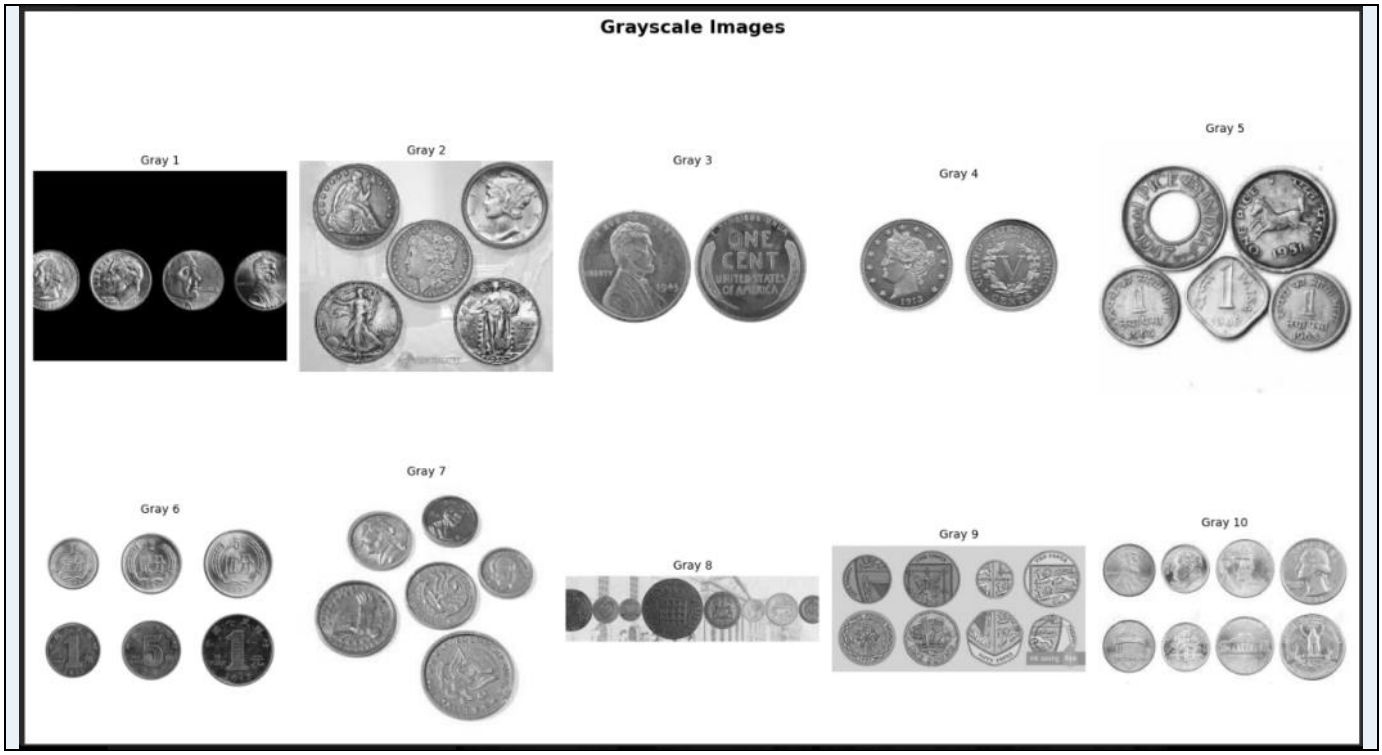


Figure: Grayscale Images

4.5 Contrast Enhancement Comparison

```
comparison_images = selected_images[:3]

plt.figure(figsize=(14, 10))
plt.suptitle("Contrast Enhancement Comparison", fontsize=16, fontweight="bold")

for i, img_name in enumerate(comparison_images):
    img = cv2.imread(os.path.join(image_path, img_name))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    hist_eq = cv2.equalizeHist(gray)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    clahe_img = clahe.apply(gray)

    plt.subplot(3, 3, 3*i + 1)
    plt.imshow(gray, cmap="gray")
    plt.title(f"Original Gray {i+1}")
    plt.axis("off")

    plt.subplot(3, 3, 3*i + 2)
    plt.imshow(hist_eq, cmap="gray")
    plt.title(f"Histogram Equalization {i+1}")
    plt.axis("off")

    plt.subplot(3, 3, 3*i + 3)
    plt.imshow(clahe_img, cmap="gray")
    plt.title(f"CLAHE {i+1}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

This cell compares grayscale images, histogram equalization, and CLAHE. CLAHE improves local contrast and is suitable for images with uneven lighting.

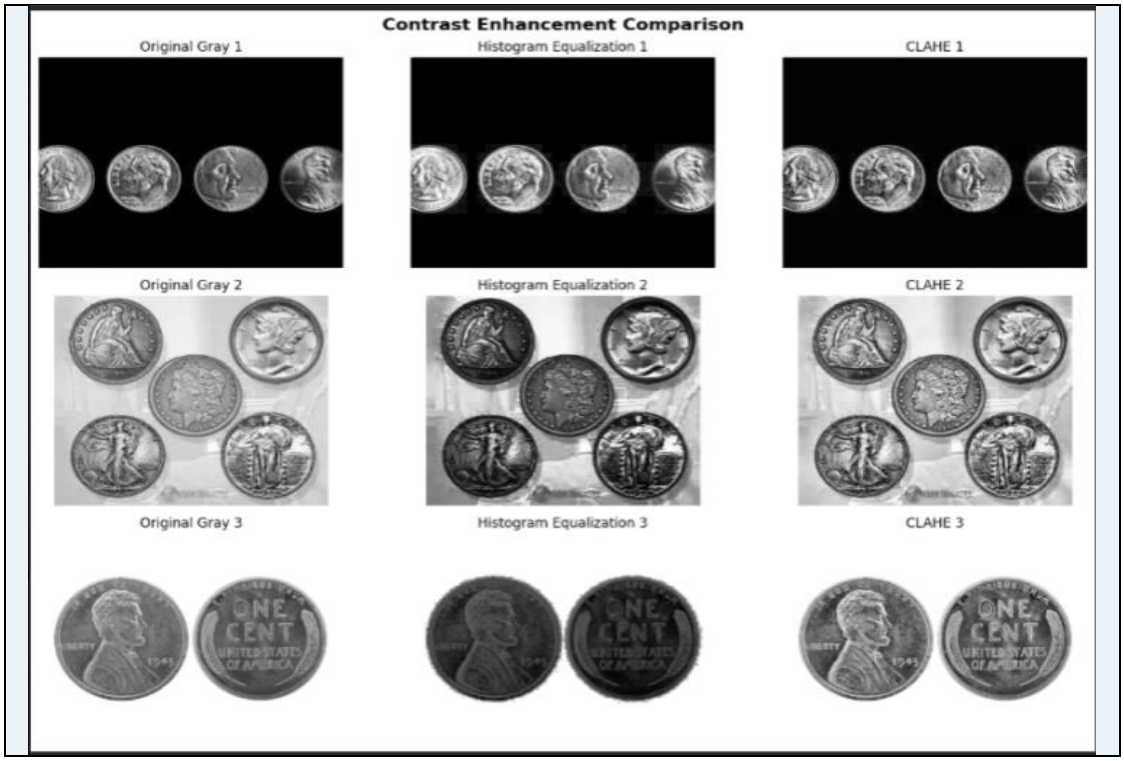


Figure: Contrast Enhancement Comparison

4.6 Smoothing Comparison

```
comparison_images = selected_images[:3]

plt.figure(figsize=(14, 10))
plt.suptitle("Smoothing Comparison: Gaussian Blur vs Median Blur", fontsize=16, fontweight="bold")

for i, img_name in enumerate(comparison_images):
    img = cv2.imread(os.path.join(image_path, img_name))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)

    gaussian = cv2.GaussianBlur(enhanced, (5, 5), 0)
    median = cv2.medianBlur(enhanced, 5)

    plt.subplot(3, 3, 3*i + 1)
    plt.imshow(enhanced, cmap="gray")
    plt.title(f"Enhanced Gray {i+1}")
    plt.axis("off")

    plt.subplot(3, 3, 3*i + 2)
    plt.imshow(gaussian, cmap="gray")
    plt.title(f"Gaussian Blur {i+1}")
    plt.axis("off")

    plt.subplot(3, 3, 3*i + 3)
    plt.imshow(median, cmap="gray")
    plt.title(f"Median Blur {i+1}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Gaussian blur reduces general noise but may blur edges. Median blur reduces noise while preserving boundaries, which is important for coin contour detection.

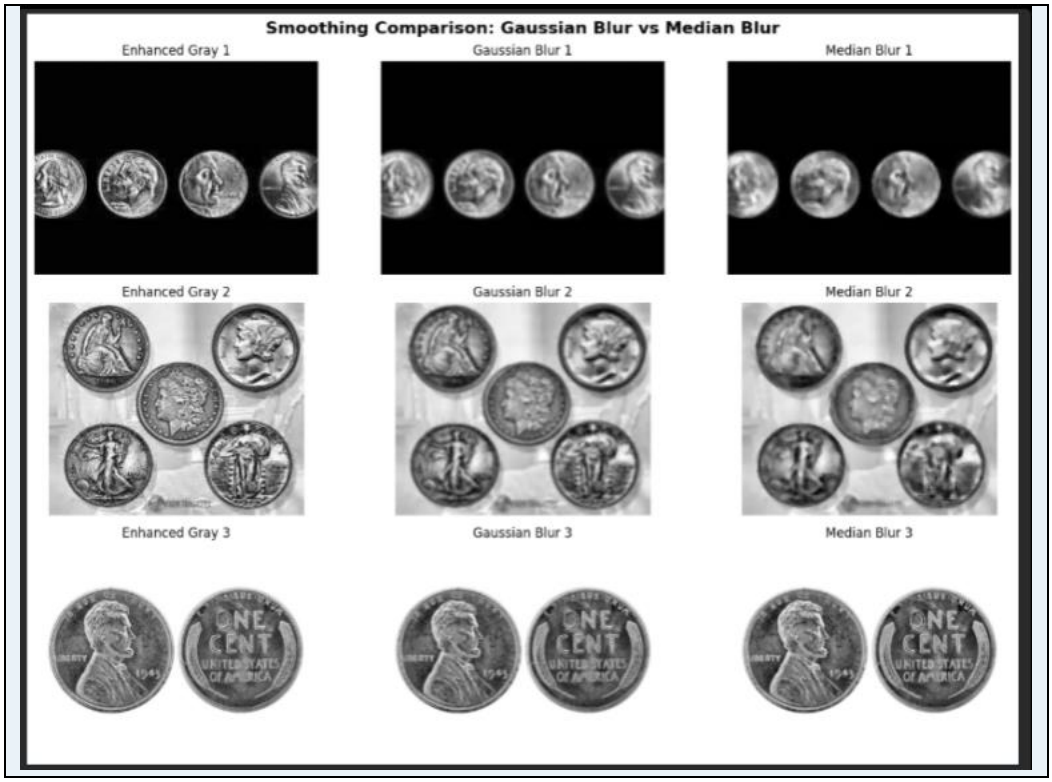


Figure: Smoothing Comparison

4.7 Thresholding Comparison

```
comparison_images = selected_images[:3]

plt.figure(figsize=(12, 9))
plt.suptitle("Thresholding Comparison: Global vs Otsu", fontsize=16, fontweight="bold")

for i, img_name in enumerate(comparison_images):
    img = cv2.imread(os.path.join(image_path, img_name))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)
    blur = cv2.medianBlur(enhanced, 5)

    _, global_th = cv2.threshold(blur, 127, 255, cv2.THRESH_BINARY_INV)
    _, otsu_th = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    plt.subplot(3, 2, 2*i + 1)
    plt.imshow(global_th, cmap="gray")
    plt.title(f"Global Threshold {i+1}")
    plt.axis("off")

    plt.subplot(3, 2, 2*i + 2)
    plt.imshow(otsu_th, cmap="gray")
    plt.title(f"Otsu Threshold {i+1}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Global thresholding uses a fixed threshold value, while Otsu thresholding automatically calculates a threshold based on the histogram. Otsu is more flexible for images with different lighting conditions.

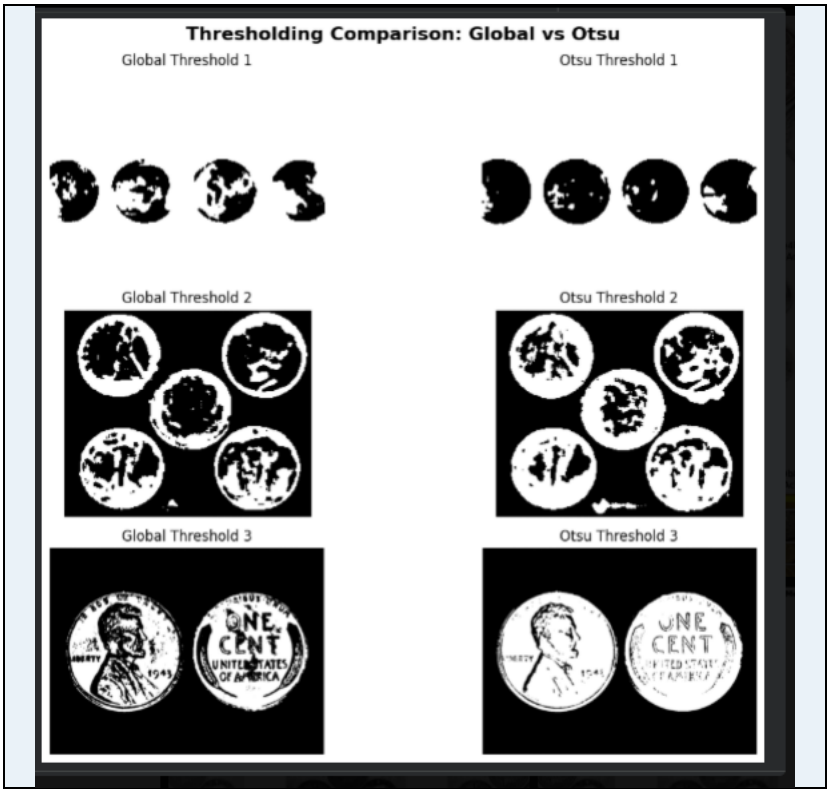


Figure: Thresholding Comparison

4.8 Morphological Operations

```
morphology_images = selected_images[:3]

plt.figure(figsize=(14, 9))
plt.suptitle("Morphological Operations: Otsu vs Opening vs Closing", fontsize=16, fontweight="bold")

for i, img_name in enumerate(morphology_images):
    img = cv2.imread(os.path.join(image_path, img_name))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)
    blur = cv2.medianBlur(enhanced, 5)

    _, th_otsu = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    kernel3 = np.ones((3, 3), np.uint8)
    kernel5 = np.ones((5, 5), np.uint8)

    opening = cv2.morphologyEx(th_otsu, cv2.MORPH_OPEN, kernel3, iterations=1)
    closing = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel5, iterations=2)

    plt.subplot(3, 3, 3*i + 1)
    plt.imshow(th_otsu, cmap="gray")
    plt.title(f"Otsu Threshold {i+1}")
    plt.axis("off")

    plt.subplot(3, 3, 3*i + 2)
    plt.imshow(opening, cmap="gray")
    plt.title(f"Opening {i+1}")
    plt.axis("off")

    plt.subplot(3, 3, 3*i + 3)
    plt.imshow(closing, cmap="gray")
    plt.title(f"Closing {i+1}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Opening removes small noise, and closing fills gaps in coin regions. These operations improve the binary image before contour detection.

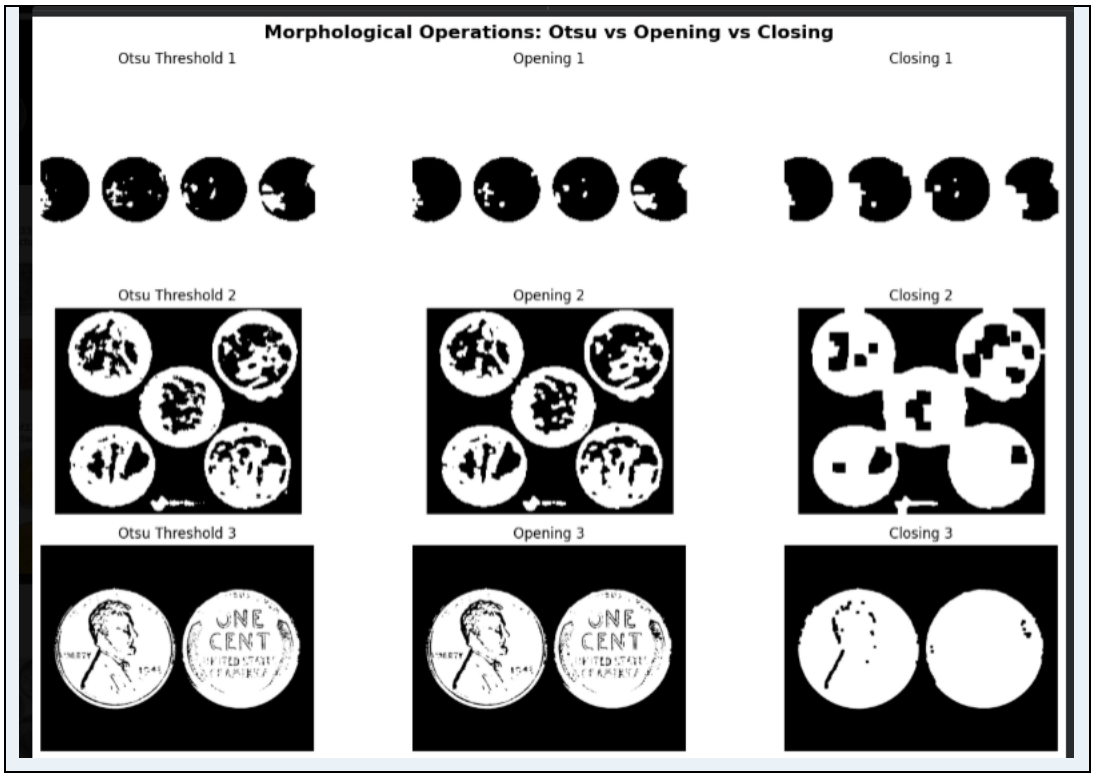


Figure: Morphological Operations

4.9 Connected Components

```
component_images = selected_images[:3]

plt.figure(figsize=(14, 8))
plt.suptitle("Connected Components Visualization", fontsize=16, fontweight="bold")

for i, img_name in enumerate(component_images):
    img = cv2.imread(os.path.join(image_path, img_name))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)
    blur = cv2.medianBlur(enhanced, 5)

    _, th = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    kernel3 = np.ones((3, 3), np.uint8)
    kernel5 = np.ones((5, 5), np.uint8)

    opening = cv2.morphologyEx(th, cv2.MORPH_OPEN, kernel3, iterations=1)
    closing = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel5, iterations=2)

    num_labels, labels = cv2.connectedComponents(closing)

    label_hue = np.uint8(179 * labels / np.max(labels))
    blank_ch = 255 * np.ones_like(label_hue)
    colored_components = cv2.merge([label_hue, blank_ch, blank_ch])
    colored_components = cv2.cvtColor(colored_components, cv2.COLOR_HSV2RGB)
    colored_components[label_hue == 0] = 0

    plt.subplot(2, 3, i + 1)
    plt.imshow(closing, cmap="gray")
```

```
    plt.title(f"Binary Image {i+1}")
    plt.axis("off")

    plt.subplot(2, 3, i + 4)
    plt.imshow(colored_components)
    plt.title(f"Connected Components {i+1}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Connected components visualize separated regions. Ideally, each coin should form one component. Touching coins may merge into one component and reduce accuracy.

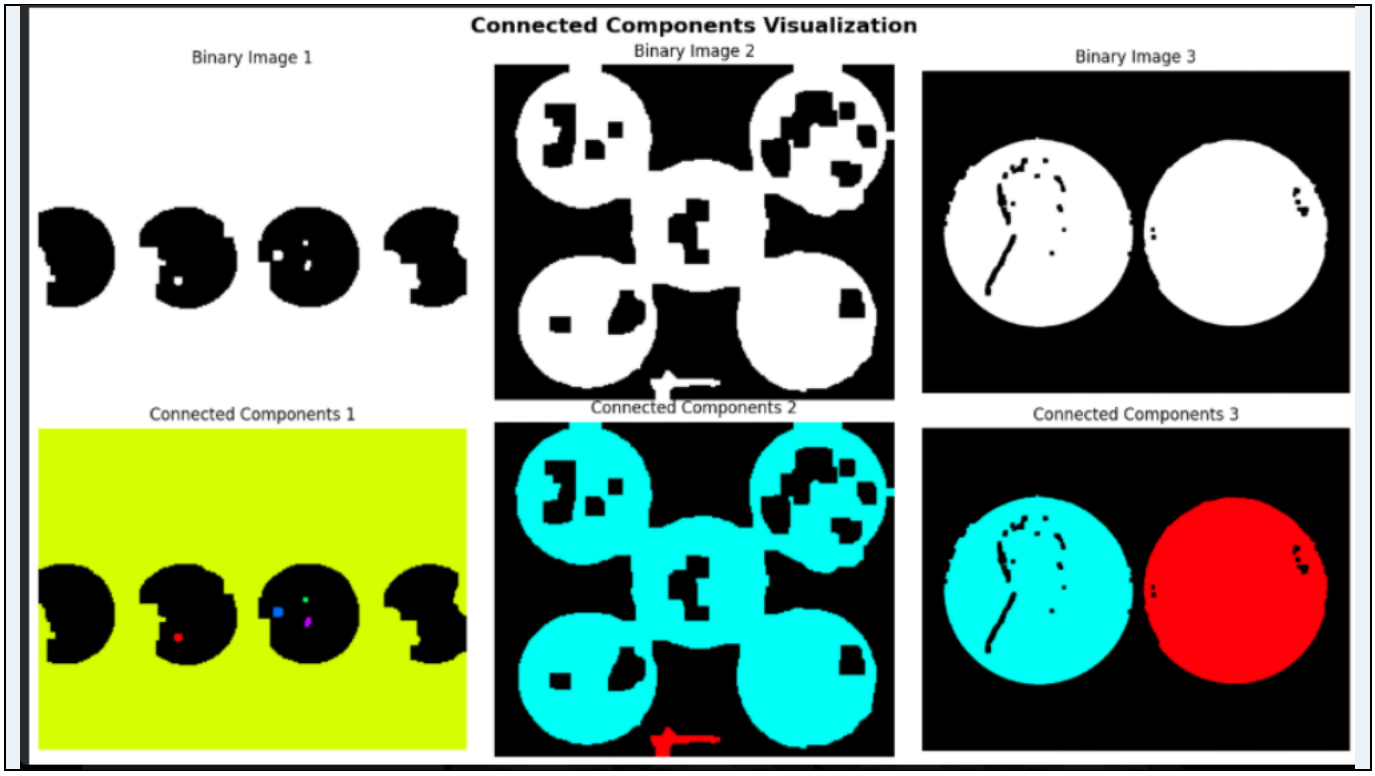


Figure: Connected Components Visualization

4.10 Main Detection Function

```
def process_coin_image(img_path):
    img = cv2.imread(img_path)

    if img is None:
        print("Error reading:", img_path)
        return None, 0, 0, []

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    h0, w0 = gray.shape
    max_size = 700
    if max(h0, w0) > max_size:
        scale = max_size / max(h0, w0)
        img_rgb = cv2.resize(img_rgb, None, fx=scale, fy=scale)
        gray = cv2.resize(gray, None, fx=scale, fy=scale)

    h, w = gray.shape
    image_area = h * w

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```
    enhanced = clahe.apply(gray)

    median = cv2.medianBlur(enhanced, 5)
    gaussian = cv2.GaussianBlur(enhanced, (5, 5), 0)

    kernel3 = np.ones((3, 3), np.uint8)
    kernel5 = np.ones((5, 5), np.uint8)

    candidates = []

    _, otsu_inv = cv2.threshold(median, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    candidates.append(("Otsu Inverse", otsu_inv))

    _, otsu_bin = cv2.threshold(median, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    candidates.append(("Otsu Binary", otsu_bin))

    adaptive_inv = cv2.adaptiveThreshold(
        gaussian, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, 51, 5
    )
    candidates.append(("Adaptive Inverse", adaptive_inv))

    edges = cv2.Canny(median, 50, 150)
    edges_closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel5, iterations=2)
    edges_filled = cv2.dilate(edges_closed, kernel3, iterations=1)
    candidates.append(("Canny Edges", edges_filled))

    def clean_and_get_contours(binary_img):
        opening = cv2.morphologyEx(binary_img, cv2.MORPH_OPEN, kernel3, iterations=1)
        closing = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel5, iterations=2)

        contours, _ = cv2.findContours(closing, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        min_area = image_area * 0.00025
        max_area = image_area * 0.30
        valid = []

        for cnt in contours:
            area = cv2.contourArea(cnt)
            if area < min_area or area > max_area:
                continue

            perimeter = cv2.arcLength(cnt, True)
            if perimeter == 0:
                continue

            circularity = 4 * np.pi * area / (perimeter ** 2)
            x, y, bw, bh = cv2.boundingRect(cnt)
            aspect_ratio = bw / float(bh)
            (cx, cy), radius = cv2.minEnclosingCircle(cnt)

            if circularity >= 0.18 and radius >= 4 and 0.35 <= aspect_ratio <= 2.8:
                valid.append(cnt)

        return valid

    best_contours = []
    best_score = -1

    for name, candidate in candidates:
        valid = clean_and_get_contours(candidate)
        count = len(valid)

        if count == 0:
            score = -100
        else:
            score = count
            if count > 25:
                score -= 15

        if score > best_score:
            best_score = score
            best_contours = valid
```

```
    valid_coins = best_contours
    output = img_rgb.copy()

    areas = [cv2.contourArea(cnt) for cnt in valid_coins]
    value_map = {"Small": 1, "Medium": 5, "Large": 10}
    total_value = 0

    if len(areas) > 0:
        small_limit = np.percentile(areas, 33)
        medium_limit = np.percentile(areas, 66)
    else:
        small_limit = 0
        medium_limit = 0

    for cnt in valid_coins:
        area = cv2.contourArea(cnt)

        if area <= small_limit:
            label = "Small"
        elif area <= medium_limit:
            label = "Medium"
        else:
            label = "Large"

        total_value += value_map[label]
        (x, y), radius = cv2.minEnclosingCircle(cnt)
        center = (int(x), int(y))
        radius = int(radius)

        cv2.circle(output, center, radius, (0, 255, 0), 2)
        cv2.putText(output, label, (center[0] - 22, center[1]),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.35, (255, 0, 0), 1)

    coin_count = len(valid_coins)
    cv2.putText(output, f"Count: {coin_count} | Total Value: {total_value}",
                (10, 18), cv2.FONT_HERSHEY_SIMPLEX, 0.35, (255, 0, 0), 1)

    return output, coin_count, total_value, valid_coins
```

This is the main function of the system. It generates multiple binary candidates, extracts contours, filters coin-like shapes, classifies coins into size categories, and calculates the total value. Filtering uses area, circularity, radius, and aspect ratio.

4.11 Final Output Display

```
display_images = selected_images[:8]
rows = len(display_images)
cols = 2

plt.figure(figsize=(14, 4 * rows))
plt.suptitle("Final Output: Original vs Detected Coins", fontsize=16, fontweight="bold")

for i, img_name in enumerate(display_images):
    img_path = os.path.join(image_path, img_name)

    original = cv2.imread(img_path)
    original_rgb = cv2.cvtColor(original, cv2.COLOR_BGR2RGB)

    output, count, total_value, valid_coins = process_coin_image(img_path)

    plt.subplot(rows, cols, 2*i + 1)
    plt.imshow(original_rgb)
    plt.title(f"Original - Image {i+1}")
    plt.axis("off")

    plt.subplot(rows, cols, 2*i + 2)
    plt.imshow(output)
    plt.title(f"Detected Count = {count}, Total Value = {total_value}")
    plt.axis("off")
```

```
plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.show()
```

This cell displays each original image beside the final result. Detected coins are circled and labeled as small, medium, or large.





Figure: Final Output: Original vs Detected Coins

4.12 Evaluation Table

```
evaluation_results = []

for img_name in selected_images:
    img_path = os.path.join(image_path, img_name)

    output, predicted_count, total_value, valid_coins = process_coin_image(img_path)

    actual_count = int(
        selected_df.loc[selected_df["image_name"] == img_name, "coins_count"].values[0]
    )

    error = abs(actual_count - predicted_count)
    accuracy = (1 - error / actual_count) * 100
    accuracy = max(0, accuracy)

    evaluation_results.append({
        "Image": img_name,
        "Actual Count": actual_count,
        "Predicted Count": predicted_count,
        "Absolute Error": error,
        "Accuracy (%)": round(accuracy, 2),
        "Total Value": total_value
    })

evaluation_df = pd.DataFrame(evaluation_results)
display(evaluation_df)

average_accuracy = evaluation_df["Accuracy (%)"].mean()
print("Average Accuracy:", round(average_accuracy, 2), "%")
```

The predicted count is compared with the actual count from the CSV file. Accuracy is calculated using the formula: Accuracy = (1 - |Actual Count - Predicted Count| / Actual Count) x 100. The project achieved approximately 74.06% average accuracy on the selected evaluation images.

4.13 Evaluation Visualizations

```
x = np.arange(len(evaluation_df))
width = 0.35

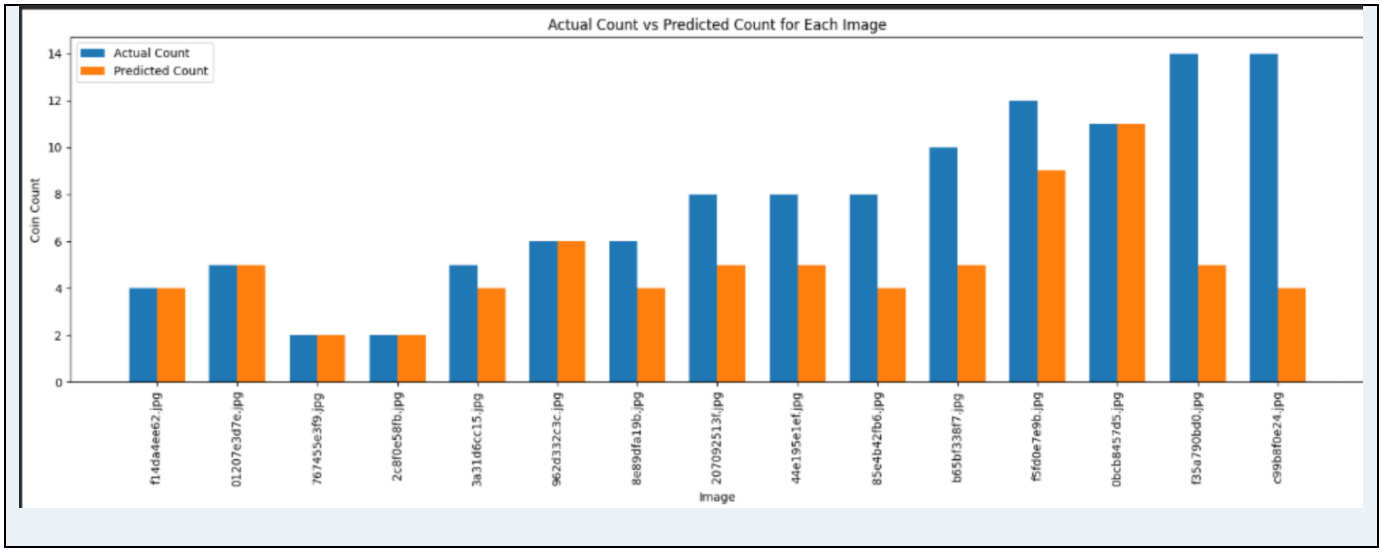
plt.figure(figsize=(16, 6))
plt.bar(x - width/2, evaluation_df["Actual Count"], width, label="Actual Count")
plt.bar(x + width/2, evaluation_df["Predicted Count"], width, label="Predicted Count")
plt.xticks(x, evaluation_df["Image"], rotation=90)
plt.xlabel("Image")
plt.ylabel("Coin Count")
plt.title("Actual Count vs Predicted Count for Each Image")
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(16, 6))
plt.bar(evaluation_df["Image"], evaluation_df["Accuracy (%)"])
plt.xticks(rotation=90)
plt.xlabel("Image")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy Percentage for Each Evaluation Image")
plt.ylim(0, 110)
plt.tight_layout()
plt.show()

mean_accuracy = evaluation_df["Accuracy (%)"].mean()
best_accuracy = evaluation_df["Accuracy (%)"].max()
worst_accuracy = evaluation_df["Accuracy (%)"].min()
mean_error = evaluation_df["Absolute Error"].mean()

print("========== Project Evaluation Summary ==========")
print("Number of Evaluation Images:", len(evaluation_df))
print("Average Accuracy:", round(mean_accuracy, 2), "%")
print("Best Accuracy:", round(best_accuracy, 2), "%")
print("Worst Accuracy:", round(worst_accuracy, 2), "%")
print("Average Absolute Error:", round(mean_error, 2))
```

These visualizations summarize the performance of the system. The first chart compares actual and predicted counts, the second chart shows accuracy for each image, and the summary statistics provide an overview of the results.



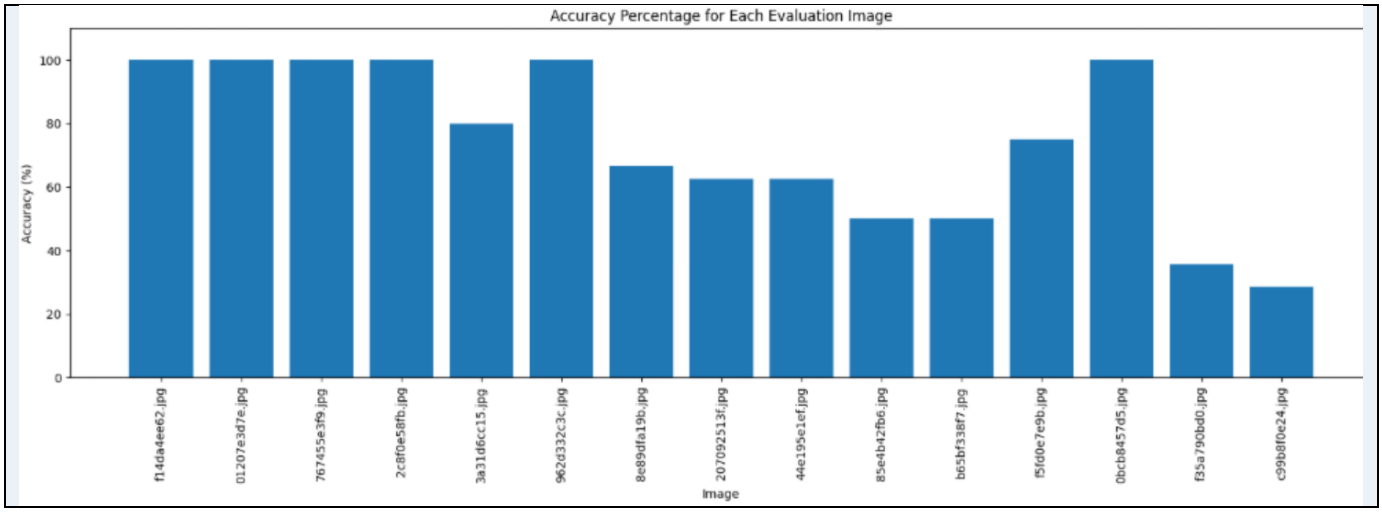


Figure: Evaluation Charts

4.14 Failure Cases

```
failure_cases = [
    "dc76e7bc74.jpg", "6d2e7e9448.jpg", "27ad5d7140.jpg",
    "4e9e203a40.jpg", "06de4d46b4.jpg", "35f856e199.jpg"
]

plt.figure(figsize=(12, 18))
plt.suptitle("Failure Cases: Difficult Coin Images", fontsize=16, fontweight="bold")

for i, img_name in enumerate(failure_cases):
    img_path = os.path.join(image_path, img_name)

    output, predicted_count, total_value, valid_coins = process_coin_image(img_path)

    actual_count = int(coins_df.loc[coins_df["image_name"] == img_name, "coins_count"].values[0])

    plt.subplot(len(failure_cases), 1, i + 1)
    plt.imshow(output)
    plt.title(f"{img_name} | Actual = {actual_count}, Predicted = {predicted_count}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Failure cases are shown separately to analyze the limitations of the system. These images include crowded, overlapping, cropped, or low-contrast coins.

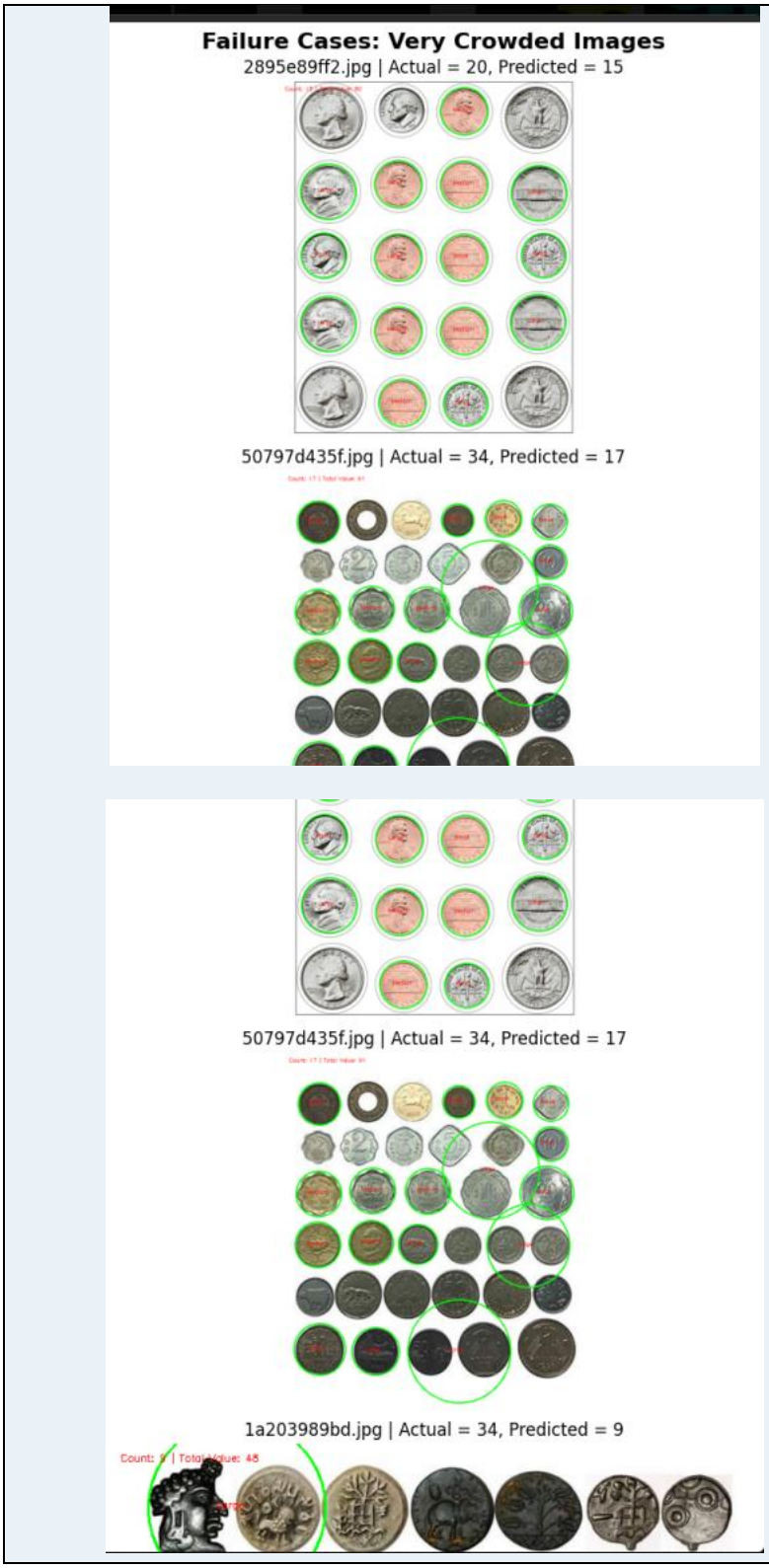


Figure: Failure Cases

6. Results and Discussion

The system achieved an average accuracy of approximately 74.06% on the selected evaluation images. The results showed that the system performed well on images with clear backgrounds, separated coins, good lighting, and circular coin shapes. Some images achieved 100% accuracy.

Lower accuracy occurred in images containing overlapping coins, cropped coins, shadows, complex backgrounds, or strong internal coin texture. These conditions made thresholding and contour detection more difficult.

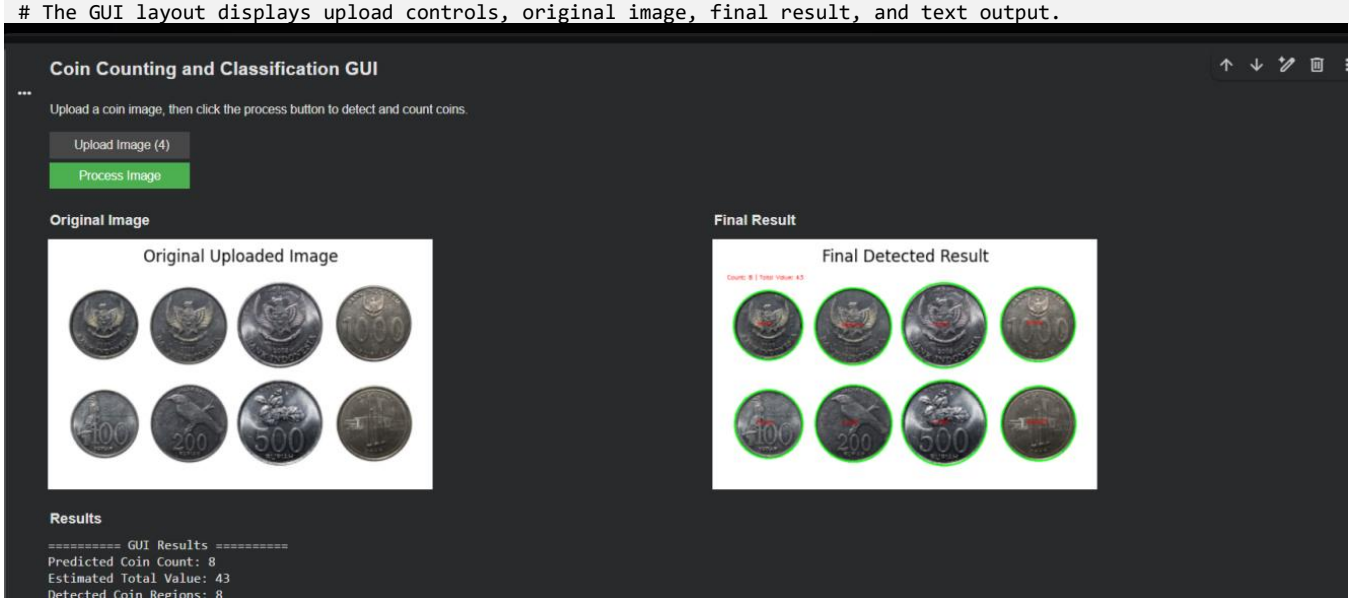
7. GUI Implementation

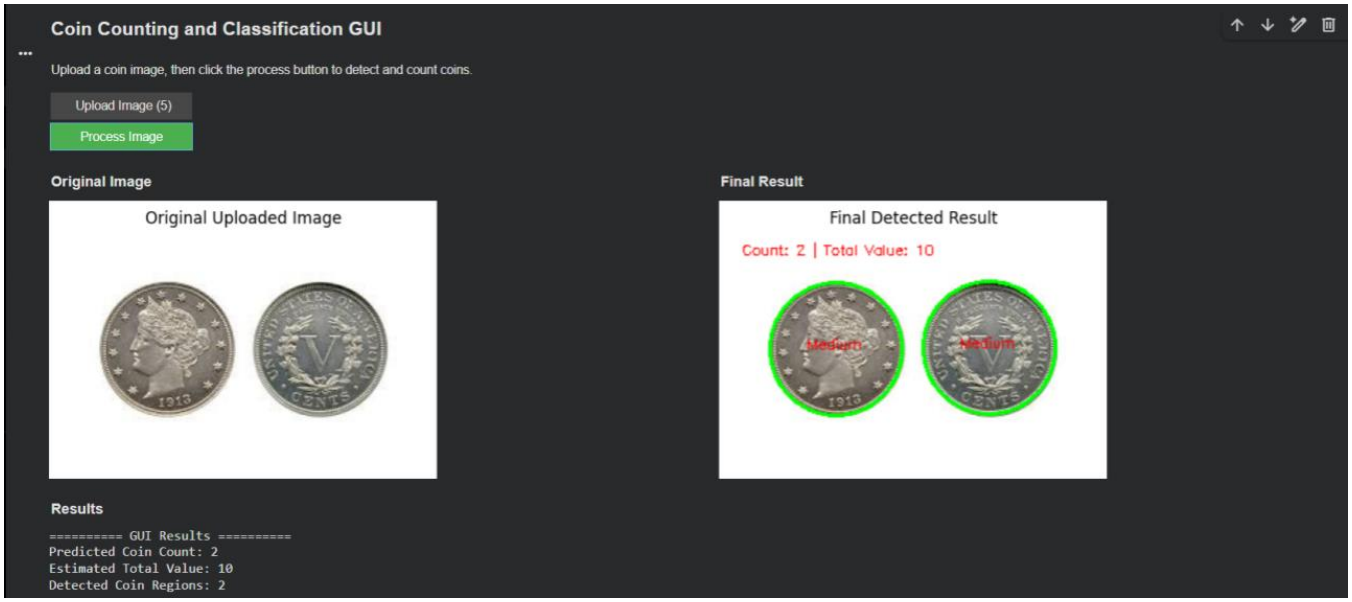
A simple graphical user interface was added as a bonus feature using ipywidgets in Google Colab. The GUI allows the user to upload an image, process it using the same coin detection pipeline, display the original image, display the final result, and show the predicted coin count and total estimated value.

```
import ipywidgets as widgets
from IPython.display import display
from PIL import Image
import io
import tempfile

upload_button = widgets.FileUpload(accept=".jpg,.jpeg,.png", multiple=False, description="Upload Image")
process_button = widgets.Button(description="Process Image", button_style="success")
original_output = widgets.Output()
result_output = widgets.Output()
text_output = widgets.Output()
uploaded_image_path = None

# The upload handler saves the uploaded image temporarily and displays it.
# The process button runs process_coin_image() and displays the final result.
# The GUI layout displays upload controls, original image, final result, and text output.
```





8. Conclusion

In this project, a coin counting and classification system was implemented using Python, OpenCV, and NumPy. The system followed a complete image processing pipeline including grayscale conversion, contrast enhancement, smoothing, thresholding, morphological operations, connected components, contour detection, and classification based on coin size.

The system successfully detected and counted coins in several images, especially when the coins were separated and the background was clear. The evaluation results showed an average accuracy of approximately 74.06% on the selected evaluation images.

The main challenges appeared in crowded images, overlapping coins, cropped coins, shadows, and complex backgrounds. Overall, the project demonstrated how classical image processing techniques can be used to solve a real-world computer vision problem.